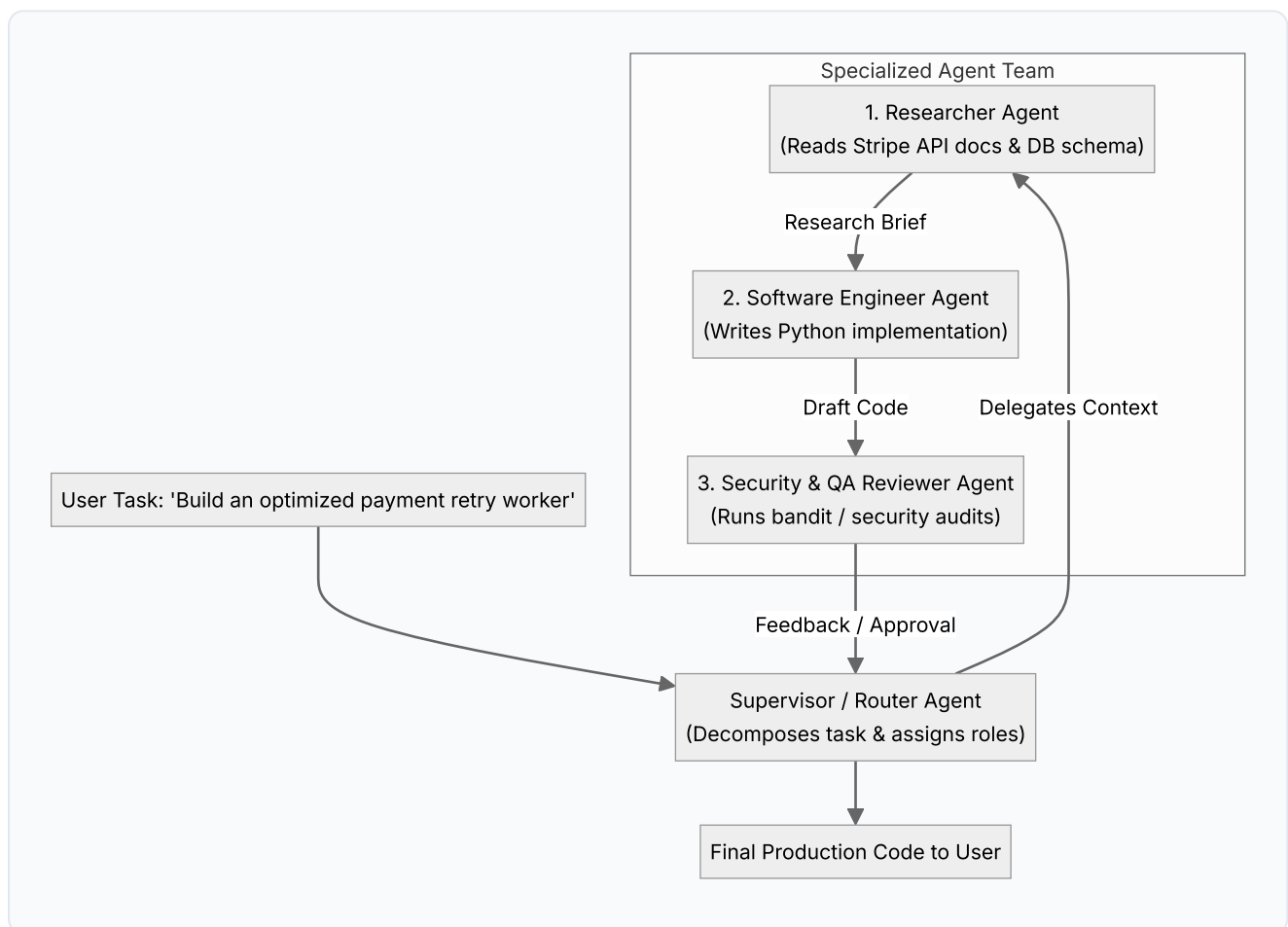


Agentic AI Chapter 7: Multi-Agent Collaboration & Production Deployment

Core Learning Objective: Master planetary-scale multi-agent systems. Build Supervisor-Worker swarms, implement peer-to-peer handoffs, stream token deltas via Server-Sent Events (SSE), and configure distributed tracing with OpenTelemetry and LangSmith.

1. Multi-Agent Topologies

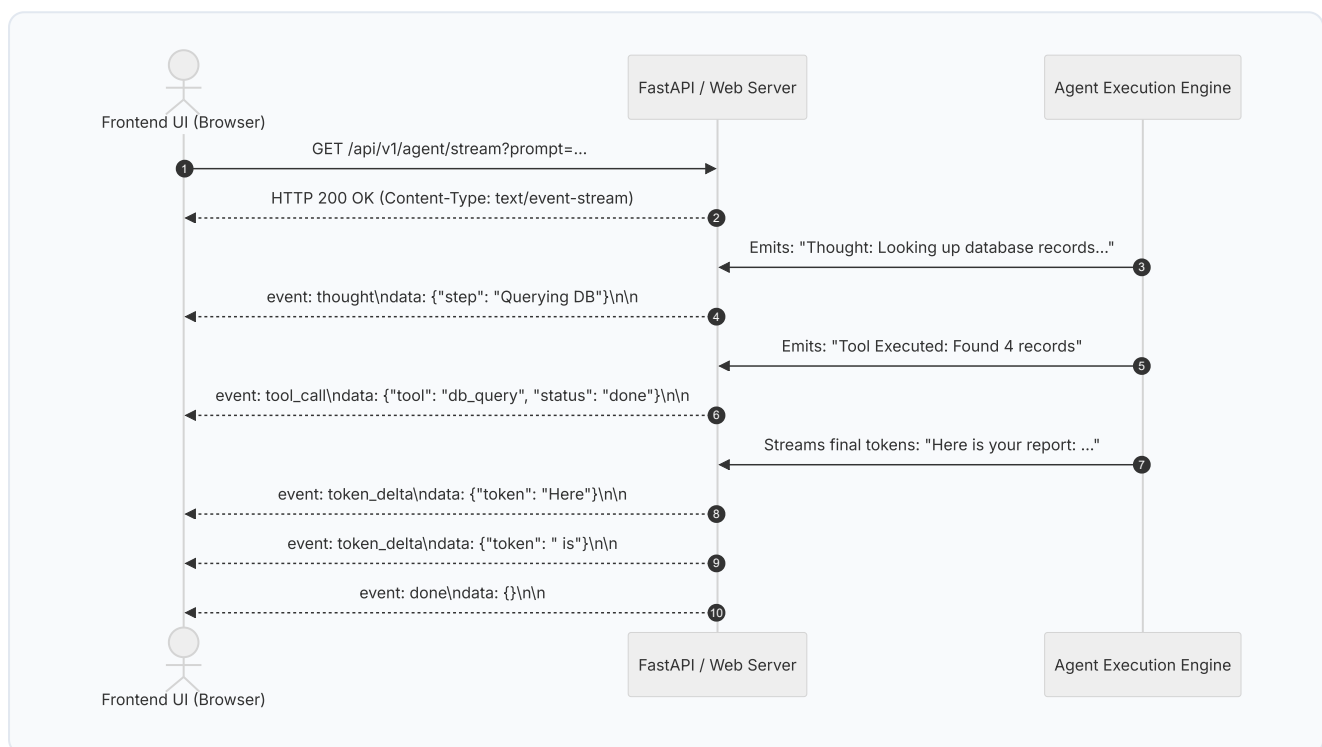
As tasks grow complex, a single monolithic agent with 30 tools fails due to context pollution and tool selection confusion. The solution is **Specialized Multi-Agent Teams**:



2. Multi-Agent Supervisor (Simulation: Scripted Agents, No Real LLM)

3. Real-Time Streaming Architecture (Server-Sent Events - SSE)

Users will not wait 30 seconds staring at a blank screen while an agent reasons and invokes tools. You must stream intermediate thoughts and token deltas in real-time using **Server-Sent Events (SSE)**:



4. Production Observability: OpenTelemetry & LangSmith

Every production agent deployment must log comprehensive execution spans:

- Trace ID:** Uniquely tracks a user request across all subagents and tools.
- Span Metrics:**
 - Exact token consumption (Prompt Tokens + Completion Tokens).
 - Wall-clock latency per LLM call vs per tool execution.
 - Model temperature, system prompt version, and exact JSON payloads.
- Error Traces:** Stack traces for flaky external tool APIs.

What the Simulation Above Does and Does Not Show

The supervisor example uses scripted agents (fixed strings) so that the control flow is deterministic and testable. It demonstrates delegation, result passing and a final merge, **not** real model behaviour. For a verified graph with checkpointing and approval see [examples/ex02_langgraph_hitl.py](#); to measure whether a multi-agent design beats a single agent, use the repeated-trial harness in [examples/ex05_eval_harness.py](#): report `pass^k` and a Wilson interval, not a single run.

Cost reality check. Multi-agent runs multiply tokens (reported around 15x a chat for research agents versus about 4x for a single agent). If the harness shows no statistically clear gain over the single-agent baseline, ship the single agent.

4. Runnable Model: A Supervisor with Handoffs, a Turn Budget and Ping-Pong Detection

Multi-agent systems fail in a characteristic way: two agents keep handing work back and forth. The supervisor below shows the three controls that prevent it.

When multiple agents help, and when one agent with tools is better

Situation	Use	Reason
Steps share most context and are strictly sequential	One agent with tools	Splitting only adds handoff cost and lost context
Subtasks are independent and parallelisable (research five companies)	Workers in parallel plus an aggregator	Wall-clock time drops and each context stays small
Different permissions are needed (read-only searcher, writer with approval)	Separate agents	Least privilege: the agent that reads untrusted web pages never holds the write tool
You want a second opinion (generate, then review)	Two agents with different prompts	Independent critique catches errors one context overlooks

Costs to say aloud. Every handoff re-sends context (tokens), every extra agent multiplies failure modes, and debugging needs a shared trace id across agents. Start with one agent; split when a measured problem (context overflow, permission separation, parallelism) demands it.

Production checklist for multi-agent systems

1. A **trace id** propagated through every agent and tool call.
 2. **Budgets per run**: turns, tokens, wall-clock, and money.
 3. **Typed handoff messages** (a schema), not free text, so a malformed handoff fails loudly.
 4. **Idempotent tools** so a retried or duplicated step does not repeat a side effect.
 5. A **human escalation path** when the supervisor hits any budget.
-

Further Reading

- [Anthropic: how we built our multi-agent research system](#)
 - [AutoGen documentation](#)
 - [OpenTelemetry documentation](#)
-

Check Yourself

Answer in your head or on paper first, then open each answer.

▼ 1. When is a single agent better than multi-agent?

When steps are tightly coupled and share lots of state; multi-agent suits broad, parallelisable work.

▼ 2. What is a handoff?

Transferring control (and chosen context) from one agent to another.

▼ 3. Why bound the number of agents and rounds?

Cost and latency multiply with agents; unbounded delegation can loop.

▼ 4. How do you decide if multi-agent helped?

Compare against a single-agent baseline with repeated trials (pass^k, confidence intervals).